Developer Activity Tracking Agent – Step-by-Step Development Guide

This guide explains **how to design and develop a Developer Activity Agent** that measures **active coding time** in tools like **IntelliJ IDEA, TOAD, Oracle APEX, and Oracle Forms**.

The approach is **enterprise-friendly**, **non-intrusive**, and suitable for **banking / regulated environments**.

STEP 1: Define Scope & Metrics (Foundation)

1.1 Decide What You Will Track

You should track **activity time**, not content.

Mandatory metrics: - Developer ID - Tool name (IntelliJ, TOAD, APEX, Forms) - Project / application name - Active time (minutes) - Idle time (minutes) - Date

Optional metrics: - File/module name - SQL execution time - Debug/run time

> ⚠️Do NOT track keystrokes or source code content.

STEP 2: High-Level Architecture

STEP 3: Database Design (Oracle)

Create a central table for activity logs.

```
CREATE TABLE dev_activity_log (
    id              NUMBER GENERATED ALWAYS AS IDENTITY,
    user_id         VARCHAR2(50),
    tool_name       VARCHAR2(30),
    project_name    VARCHAR2(100),
    activity_state  VARCHAR2(10), -- ACTIVE / IDLE
    start_time      TIMESTAMP,
    end_time        TIMESTAMP,
    duration_min    NUMBER,
    created_date    TIMESTAMP DEFAULT SYSTIMESTAMP
);
```

STEP 4: Build Central Backend (Spring Boot)

4.1 Create REST API

Endpoint:

```
POST /api/dev-activity
```

Payload:

```
{
  "userId": "DEV001",
  "tool": "INTELLIJ",
  "project": "LoanApp",
  "state": "ACTIVE",
  "startTime": "2025-01-10T10:30:00",
  "endTime": "2025-01-10T11:00:00"
}
```

4.2 Persist Data

- Validate payload
- Calculate duration
- Insert into Oracle DB

STEP 5: Develop Local Agent (Core Logic)

This agent runs on the **developer's machine**.

5.1 Choose Technology

Recommended: - Java 17+ - JNA (Windows activity detection) - Runs as background service

---

STEP 6: Detect Active Application

6.1 Window Focus Detection

Logic:

```
IF foreground window belongs to known dev tool
THEN mark tool as ACTIVE
ELSE mark as INACTIVE
```

Map process names:

Tool	Process Name
IntelliJ	idea64.exe
TOAD	toad.exe
Oracle Forms	ifrun60.exe
Browser (APEX)	chrome.exe / msedge.exe

STEP 7: Idle Detection Logic

7.1 Idle Rule

```
IF no keyboard/mouse input > 5 minutes
THEN state = IDLE
```

Idle time is **excluded** from coding time.

STEP 8: Tool-Specific Tracking

8.1 IntelliJ IDEA (Plugin – Best Accuracy)

Steps: 1. Create IntelliJ Plugin project 2. Listen to editor events 3. Track file editing time 4. Send activity to backend

You can track: - Typing activity - File open duration - Project name

---

8.2 TOAD for Oracle

Approach: - OS-level focus detection - Mouse/keyboard activity - Optional: parse TOAD logs

Limitation: No internal API

8.3 Oracle APEX

Approach: 1. Add JavaScript hooks 2. Track page load/save events 3. Send REST calls

Example JS:

```
setInterval(() => {
  fetch('/ords/dev/activity', { method: 'POST' });
}, 60000);
```

8.4 Oracle Forms

Approach: - Track runtime window focus - Track activity time only

Limitation: No internal instrumentation

STEP 9: Time Calculation Logic

```
Start timer → Tool focused & active
Pause timer → Idle OR window unfocused
Resume timer → Activity resumes
```

Only **ACTIVE duration** is stored.

STEP 10: Data Upload Strategy

10.1 Batch Upload (Recommended)

- Store locally
- Push every 5–10 minutes
- Retry on failure

Avoid real-time chatty calls.

STEP 11: APEX Dashboard

Suggested dashboards: - Developer vs Coding Hours - Tool-wise Time Split - Daily / Weekly Trends - Idle vs Active Ratio

Example query:

```
SELECT tool_name,
       SUM(duration_min)/60 AS hours
FROM dev_activity_log
GROUP BY tool_name;
```

STEP 12: Security & Compliance

✔️Inform developers clearly ✔️No keystroke/content capture ✔️Encrypt REST communication ✔️Role-based access

---

STEP 13: Testing Strategy

- Unit test agent logic
- Simulate idle scenarios
- Verify tool switching
- Validate dashboard numbers

STEP 14: Deployment

- Package agent as installer
- Auto-start on login
- Version control updates

STEP 15: MVP Timeline

Phase	Duration
Backend API	4–5 days
IntelliJ Plugin	5–7 days
OS Agent	7–10 days
APEX Dashboard	3–4 days

Total: ~3–4 weeks

Final Recommendation

Start with: 1. IntelliJ Plugin 2. APEX Tracking 3. Central API + Dashboard

Then extend to TOAD & Oracle Forms.

If you want next: - IntelliJ plugin **sample code** - Java **Windows activity detector** - APEX **dashboard SQLs** - BRD / SRS document

Tell me what you want to build next 🚀